

- Data: 04/09/2025

## GABARITO - Prova SUBSTITUTIVA de Linguagens de Programação

### 1: Considere as especificações/definições abaixo.

1. Os caracteres que são reconhecidos como identificadores válidos.
2. O efeito de se comparar duas strings usando o operador `==`.
3. Que um bloco de comandos (exemplo: dentro de um `for`) é especificado por `{`.
4. O modelo de implementação da linguagem (compilada, interpretada, híbrida)
5. O valor inicial de uma variável não inicializada pelo programador.
6. O que é realizado quando o operador `>>` é aplicado a uma variável.
7. Como comentários são definidos e reconhecidos em um programa.
8. Como caracteres de escape (por exemplo `\n` para nova linha) são especificados dentro de uma string.
9. Número de bytes de um tipo inteiro.
10. As palavras reservadas da linguagem.

Quais delas poderiam fazer parte da SEMÂNTICA de uma linguagem?

- a. ☐ 1, 3
- b. ☐ 1, 3, 7, 8, 10
- c. ☐ 1, 3, 7, 10
- d. ☐ **X** 2, 5, 6, 9
- e. ☐ 2, 5, 6, 9, 10
- f. ☐ 2, 6

### 2: Considere a seguinte regra BNF:

```
<if> -> if <expr> then <stmt> else <stmt>
```

Quais dos seguintes códigos estão de acordo com essas regras BNF, supondo que `(a < 0)` satisfaz a regra de `<expr>` e tanto `a++`; e `b++`; satisfazem a regra `<stmt>`?

1. `if (a < 0) then a++; else b++;`
  2. `if (a < 0) then a++;`
  3. `if (a < 0) then b++;`
- a. ☐ **X** Apenas o código 1
  - b. ☐ Apenas o código 2
  - c. ☐ Apenas o código 3
  - d. ☐ Os códigos 2 e 3
  - e. ☐ Todos os códigos de 1 a 3
  - f. ☐ Só é possível afirmar após a avaliação da expressão `(a < 0)`

### 3: Considere as seguintes regras BNF

```
<attr> -> <id> := <expr>
<id>    -> A | B | C
<expr> -> <expr> + <expr>
        | <expr> * <expr>
        | ( <expr> )
        | <id>
```

Considere as seguintes árvores de análise **K**, **J**, **F**, **T**.

Qual destas árvores corresponde à derivação da sentença para o código `A := B * (A + C)`?

- a. ☐ F
- b. ☐ J
- c. ☐ K
- d. ☐ **X** T
- e. ☐ Quaisquer árvores são válidas
- f. ☐ Nenhuma das árvores é válida

4: Em Java, para uma variável que é um objeto, o local de armazenagem e o amarração de armazenagem é:

- a. ☐ X Heap - Dinâmico
- b. ☐ Heap - Estático
- c. ☐ Pilha - Dinâmico
- d. ☐ Pilha - Estático
- e. ☐ Área Estático - Dinâmico
- f. ☐ Área Estático - Estático

5: Considere a implementação em C abaixo da função `soma_vetores` que soma os elementos de dois vetores

```
1  int *soma_vetores(int v1[], int v2[], int dim) {
2      int v_soma[dim];
3      int i = 0;
4      for (i = 0; i < dim; i++) {
5          v_soma[i] = v1[i] + v2[i];
6      }
7      return v_soma;
8  }
```

Em qual área da memória feita a amarração de armazenagem dos vetores referenciados por `vet1`, `vet2` e `v_soma`, respectivamente?

- a. ☐ Ocorre na pilha para todos os vetores.
- b. ☐ Ocorre no heap para todos os vetores.
- c. ☐ Depende do código que invoca `soma_vetores` para todos os vetores.
- d. ☐ X Depende do código que invoca `soma_vetores`, para `vet1`, `vet2`. Para `v_soma` ocorre na pilha.
- e. ☐ Depende do código que invoca `soma_vetores`, para `v_soma`. Para `vet1`, `vet2` ocorre na pilha.
- f. ☐ Ocorre na área de armazenamento estático para todas as variáveis, exceto os parâmetros da função.

6: Considere o seguinte trecho de código C

```
1  int i = 0;
2
3  int f(int j) {
4      if (j == 0) {
5          i++;
6          return 0;
7      }
8      return j % 2;
9  }
```

Para os seguintes aspectos de 1 a 4, qual é momento de amarração/vinculação?

- 1. Significado do operador `==`
  - 2. Implementação da operação `(j == 0)`
  - 3. Amarração do endereço de `j`
  - 4. Amarração do endereço de `i`
- a. ☐ (1) Definição da linguagem, (2) compilação, (3) execução, (4) execução
  - b. ☐ (1) Definição da linguagem, (2) compilação, (3) carga, (4) carga
  - c. ☐ (1) Definição da linguagem, (2) definição da linguagem, (3) execução, (4) carga
  - d. ☐ (1) Definição da linguagem, (2) execução, (3) execução, (4) carga
  - e. ☐ X (1) Definição da linguagem, (2) compilação, (3) execução, (4) carga
  - f. ☐ (1) compilação, (2) compilação, (3) execução, (4) carga

7: Em gerenciamento automático de memória, um espaço na heap deve ser desalocado quando

- a. ☐ for feita uma nova atribuição a uma variável (que anteriormente referenciava outro dado em memória)
- b. ☐ o escopo de uma variável terminar.
- c. ☐ quando o respectivo bloco de execução terminar.
- d. ☐ X quando não for possível acessar o dado da heap em memória por uma variável ativa.
- e. ☐ quando uma variável referenciar um espaço de memória heap já referenciado por outra variável de heap.
- f. ☐ ao término da execução de um programa ou subprograma.

8: No fim do código C abaixo, caso tenha terminado o escopo de todas as variáveis, deve-se

```
1 ptr_a = malloc(int)
2 ptr_b = malloc(int)
3 a = 33
4 b = 11
5 ptr_b = &b
6 *ptr_b = b/11
7 *ptr_a = a/(*ptr_b)
```

- a. ☐ X Liberar o espaço em memória apontado unicamente por prt\_a.
- b. ☐ Liberar o espaço em memória apontado unicamente por prt\_a e prt\_b.
- c. ☐ Liberar o espaço em memória apontado unicamente por prt\_a e prt\_b, e da variável b.
- d. ☐ Liberar o espaço em memória apontado unicamente por prt\_a e prt\_b, e das variáveis a e b.
- e. ☐ Liberar o espaço em memória apontado unicamente por a e b.
- f. ☐ Não há nenhum requisito de liberação de memória no código indicado

9: Considere o código Java abaixo.

```
1 class L {
2     public String id;
3     public L prev;
4     public L next;
5 }
6
7 public static void f(L l4) {
8     L l1, l2, l3;
9     l1 = new L(); l1.id = "obj1";
10    l2 = new L(); l2.id = "obj2";
11    l3 = new L(); l3.id = "obj3";
12    l1.next = l2;
13    l2.next = l3;
14    l3.next = l2; // quantas referências aqui!
15    l4.next = l1; // ultima
16 }
```

Ao final de uma invocação f(1), quais objetos em memória podem ser desalocados pelo coletor de lixo?

- a. ☐ l1
- b. ☐ l2
- c. ☐ l3
- d. ☐ l1, l2, l3
- e. ☐ l1, l2, l3, l4
- f. ☐ X nenhum

10: Quando uma linguagem oferece expressões com avaliação em curto-circuito?

- a. ☐ Quando, ao avaliar uma expressão, ocorre overflow ou underflow.
- b. ☐ X Quando, ao avaliar uma expressão, dependendo da avaliação de uma porção da expressão o restante da expressão deixa de ser avaliada por não influir no resultado final da avaliação.
- c. ☐ Quando, ao avaliar uma expressão, uma linguagem deixa de avaliar uma expressão que poderia causar erros.
- d. ☐ Quando, ao avaliar uma expressão, as subexpressões são avaliadas isoladamente e o resultado final apenas considera os valores relevantes das subexpressões.
- e. ☐ Quando, ao avaliar uma expressão, a otimização do compilador simplifica as expressões.
- f. ☐ Todas as demais explicações para curto-circuito estão corretas.

11: Considere o seguinte programa C:

```
1 int fun(int *i) {
2     *i += 5;
3     return 4;
4 }
5
6 void main() {
```

```

7     int x = 3;
8     x = x + fun(&x);
9 }

```

Qual é o valor de x após a última atribuição em `main()` considerando as seguintes opções:

1. Operandos são avaliados da esquerda para a direita (E->D)
  2. Operandos são avaliados da direita para a esquerda (D->E)
- a. ☐ E->D: 8 - D->E: 8
  - b. ☐ E->D: 7 - D->E: 7
  - c. ☐ E->D: 8 - D->E: 7
  - d. ☐ E->D: 7 - D->E: 8
  - e. ☐ **X** E->D: 7 - D->E: 12
  - f. ☐ E->D: 12 - D->E: 12

**12: O que é a instância do registro de ativação (ARI) em subprogramas?**

- a. ☐ É uma entrada na pilha de execução de um subprograma.
- b. ☐ É uma estrutura de dados que armazena os dados necessários para realização de uma chamada de subprograma e o consequente retorno ao código que o invocou.
- c. ☐ É uma estrutura de dados que armazena as variáveis locais de um subprograma.
- d. ☐ É uma estrutura de dados que indica quando um subprograma foi invocado.
- e. ☐ **X** É o conjunto de dados de descreve uma chamada particular de subprograma, indicando o endereço de retorno ao código que o invocou, suas variáveis locais e parâmetros.
- f. ☐ É o conjunto de dados locais de um subprograma em execução.

**13: Considere as seguintes propostas de uso de parâmetro variático em uma função Python.**

1. `def f(a,b,c, ...)`
2. `def f(a,b,c,*d)`
3. `def f(a,b,c,*d,**e)`
4. `def f(a,b,c,**d,**e)`

Quais das sintaxes Python de uso de parâmetro variático são **INVÁLIDAS**?

- a. ☐ apenas 1
- b. ☐ apenas 2
- c. ☐ apenas 3
- d. ☐ apenas 4
- e. ☐ **X** 1 e 4
- f. ☐ todas são inválidas

**14: Considere o código C a seguir, que não compila por possuir um erro da linha 5.**

```

1  #include <stdio.h>
2
3  int v[3][5] = {{1,2,3,4,5},{6,7,8,9,10},
4                {11,12,13,14,15}};
5
6  void imprime(int v[][]) { // Erro de compilação
7      printf("v[1][1]=%d ; v[2][1]=%d",v[1][1],v[2][1]);
8  }
9
10 int main() {
11     imprime(v);
12     return 0;
13 }

```

Para corrigir o código, realizando o MÍNIMO de modificações requeridas, deve-se substituir a linha indicada por:

- a. ☐ `void imprime(int v[][3]) {`
- b. ☐ **X** `void imprime(int v[][5]) {`
- c. ☐ `void imprime(int v[3][]) {`
- d. ☐ `void imprime(int v[5][]) {`
- e. ☐ `void imprime(int v[3][5]) {`

```
f. [ ] void imprime(int *v) {
```

Para as questões seguintes, considere a declaração de classes abaixo e a declaração e inicialização de objetos indicados no método main.

```
1  class Baggio {
2      private:
3          int _baggio;
4      public:
5          Baggio(int baggio) {
6              this->_baggio = baggio; }
7          int m1() {
8              return 1;
9          }
10         virtual int m2() {
11             return 2;
12         }
13         int getBaggio() {
14             return _baggio;
15         }
16     };
17
18     class Pilao : public Baggio {
19     private:
20         int _pilao;
21     public:
22         Pilao(int baggio, int pilao) : Baggio(a) {
23             _pilao = pilao;
24         }
25         int m1() {
26             return 3;
27         }
28         int m3() {
29             return 4;
30         }
31         int getPilao() { return _pilao; }
32     };
33
34     class Orfeu : public Pilao {
35     private:
36         int _orfeu;
37     public:
38         Orfeu(int baggio, int pilao, int orfeu) :
39             Pilao(a, pilao) {
40             this->_orfeu = orfeu;
41         }
42         int m2() {
43             return 5;
44         }
45         int getOrfeu() { return _orfeu; }
46     };
47
48     class Melitta : private Baggio {
49     private:
50         int _melitta;
51     public:
52         Melitta(int baggio, int melitta) : Baggio(a) {
53             this->_melitta = melitta;
54         }
55         int m2() {
56             return 6;
57         }
58     };
```

```

58     int getMelitta() { return _melitta; }
59 };
60
61 int main() {
62
63     Baggio baggio(1), *ptr_baggio;
64     Pilao pilao(2,3), *ptr_pilao;
65     Orfeu orfeu(4,5,6), *ptr_orfeu;
66     Melitta melitta(7,8), *ptr_melitta;
67
68     ptr_baggio = new Baggio(1);
69     ptr_pilao = new Pilao(2,3);
70     ptr_orfeu = new Orfeu(4,5,6);
71     ptr_melitta = new Melitta(7,8);
72
73 }

```

**15:** Considerando as inicializações indicadas, quais pares de sentenças C++ que, isoladamente, produzem erro de compilação? Observe que basta ocorrer erro de compilação em uma das sentenças.

- a. ☐ Baggio baggio2 = orfeu; Baggio \*ptr\_baggio2 = new Orfeu(4,5,6);
- b. ☐ orfeu.m2(); orfeu.m1();
- c. ☐ baggio.m1(); pilao.m2();
- d. ☐ Baggio \*ptr\_baggio2 = new Orfeu(4,5,6); pilao.m2();
- e. ☐ **X** Baggio \*ptr\_baggio3 = new Melitta(7,8); orfeu.m1();

**16:** Para a questão anterior, qual é a origem do erro de compilação encontrado?

- a. ☐ violação de regra de atribuição entre instâncias de classes
- b. ☐ acesso a membro (atributo ou método) privado
- c. ☐ **X** herança privada
- d. ☐ membro (atributo ou método) inexistente
- e. ☐ polimorfismo não se aplica na invocação
- f. ☐ outro motivo

**17:** Considerando as mesmas inicializações indicadas no método main, quais pares de sentenças C++ compilam corretamente? Ambas as sentenças devem compilar.

- a. ☐ **X** baggio.m1(); ptr\_melitta->m2();
- b. ☐ pilao.m1(); melitta.m1();
- c. ☐ ptr\_melitta->m1(); ptr\_orfeu = &baggio;
- d. ☐ Baggio \*ptr\_baggio3 = new Melitta(7,8);
- e. ☐ baggio.getPilao(); ptr\_baggio->m1();
- f. ☐ ptr\_orfeu = &baggio; ptr\_melitta->m2();

**18:** O CIR do objeto ptr\_pilao aponta para uma tabela virtual de classes (vtable) que

- a. ☐ É da classe Baggio e contém um único ponteiro para o método m2().
- b. ☐ É da classe Baggio e contém três ponteiros de métodos m2(), getBaggio() e getPilao().
- c. ☐ É da classe Baggio e contém um único ponteiro para o método getBaggio().
- d. ☐ **X** É da classe Pilao e contém um único ponteiro para o método m2().
- e. ☐ É da classe Pilao e contém três ponteiros de métodos: m1(), m3() e getPilao().
- f. ☐ É da classe Pilao e contém um único ponteiro para o método getPilao().